Machine Learning and Deep Learning
Lecture-10

By: Somnath Mazumdar
Assistant Professor
sma.digi@cbs.dk

Spring 2026

Topics

- Distributed Deep learning
- TensorFlow
- Recurrent Neural Networks (RNNs)
- RNN Variants:
  - LSTM
  - GRU
  - Bidirectional LSTM

Distributed DL

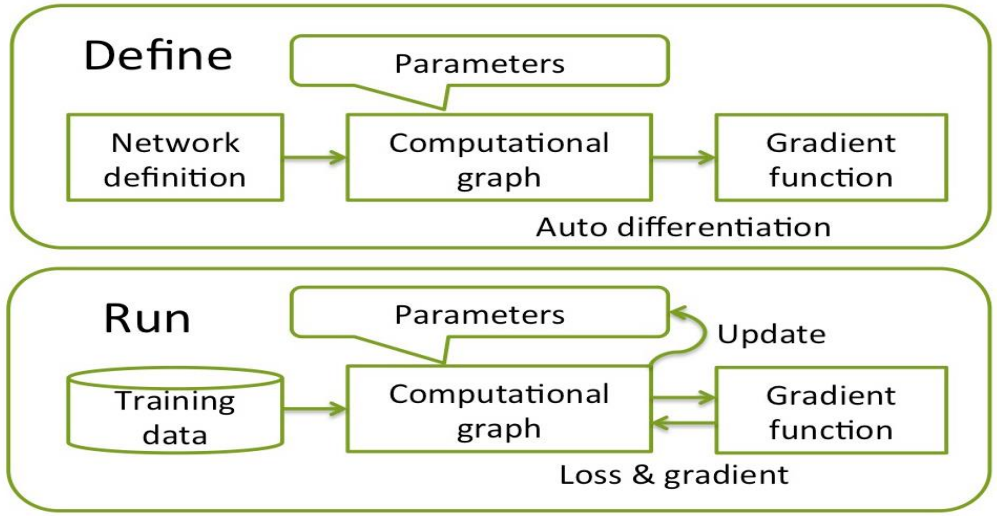
DL Framework

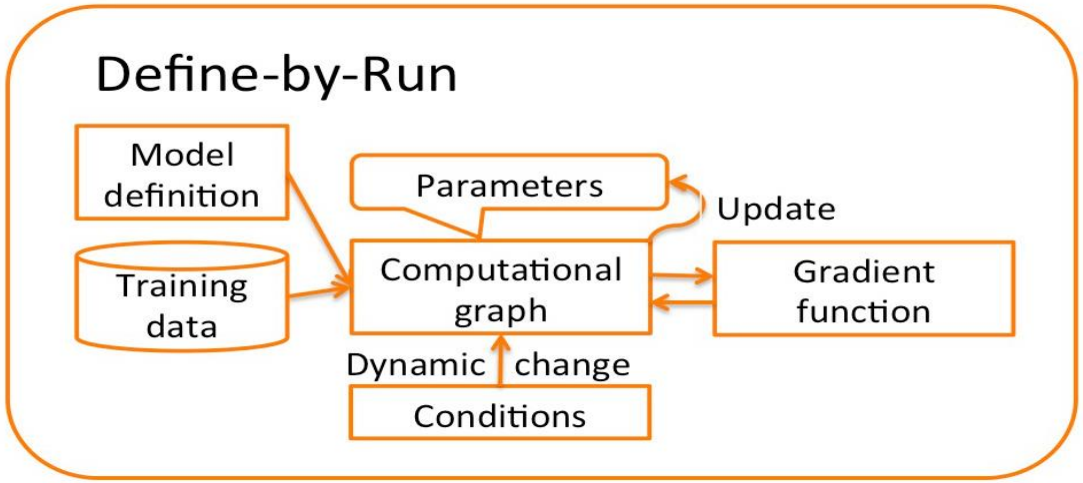
- It abstracts much of the underlying mathematics and automatic differentiation, allowing to focus on:
  - Architecture design
  - Model training and optimization
- They provide:
  - Acceleration
  - Automatic backpropagation
  - Pre-built layers and loss functions

- Evolution of Larger and Deeper Models
  - LeNet (1998): One of the earliest CNNs
  - AlexNet (2012): Breakthrough in ImageNet competition
  - ResNet (2015): Enabled very deep networks (e.g., ResNet-50, ResNet-101)
  - Transformer (2017): Introduced self-attention mechanism
    - Foundation for modern BERT, GPT

Model Training

- Steps: 1) build a computational graph from network definition, 2) input training data and compute loss function, 3) update parameters.





- Define-and-run: DL frameworks complete step one in advance of step two (TensorFlow, Caffe).

- Define-by-run: Combines steps one and two into a single step (PyTorch).
  - Computational graph is not given before training but obtained while training.

https://www.oreilly.com/content/complex-neural-networks-made-easy-by-chainer/

Good to know....ONNX

- ONNX: Open Neural Network eXchange (https://onnx.ai/)
- Open-source shared model representation for framework interoperability and shared optimization.
- ONNX defines a common set of operators, data types and a common file format to enable developers to use models with a variety of frameworks, tools, runtimes, and compilers.
- ONNX provides a definition of an extensible computation graph model
- TensorFlow supports ONNX via export and conversion tools.

Non-distributed ML Way

- Standard way to execute ML model

$$\text{argmin}_{\boldsymbol{x}} f(\boldsymbol{x})$$

Solved via **optimization procedure**, e.g. **stochastic gradient descent** (SGD).

$$f(x) = \sum_{i=1}^{M} loss(x, exi)$$

Notion of "quality," e.g. **squared distance**

$$\text{model } \boldsymbol{x}$$

E.g., **neural network** or **linear model**

Task

E.g., **classification**

Data

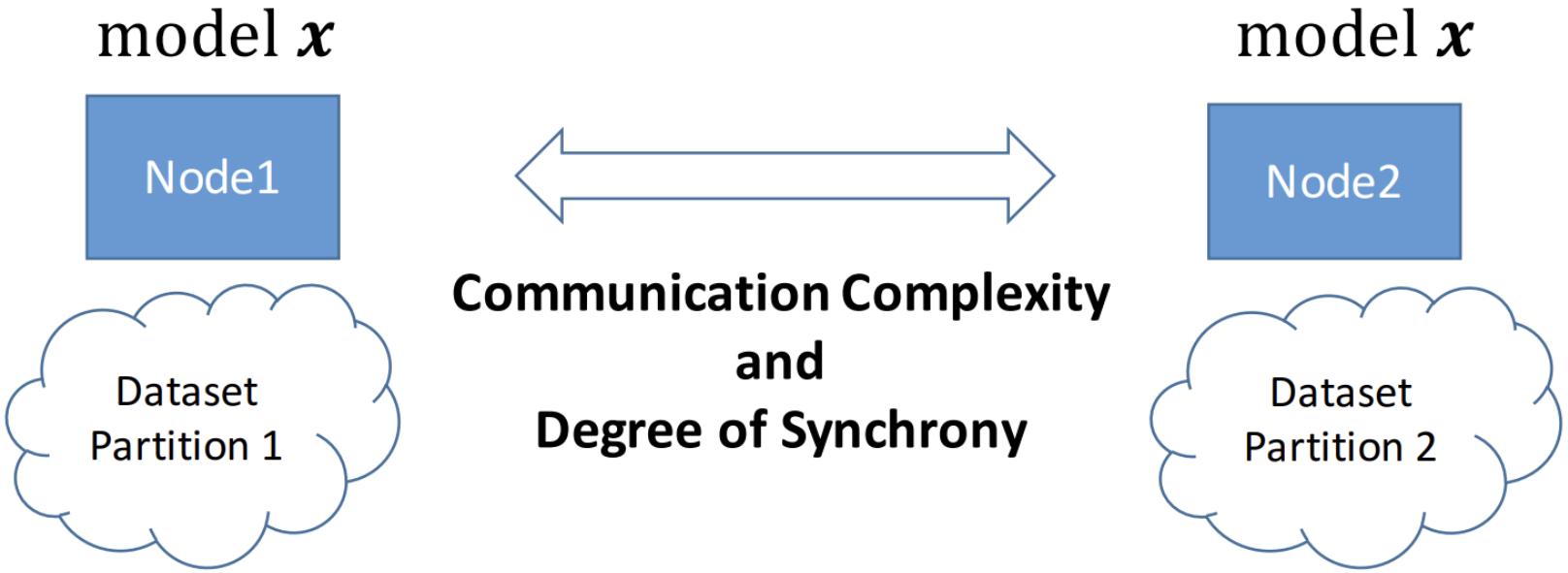
Distributed ML Way

- (Standard) data parallel paradigm, but there are also model parallel or hybrid approaches.

$$\text{argmin}_{\boldsymbol{x}} f(\boldsymbol{x}) = f_1(\boldsymbol{x}) + f_2(\boldsymbol{x})$$

$$f_1(x) = \sum_{i=1}^{M/2} l(x, ei)$$

$$f_2(x) = \sum_{i=\frac{M}{2}+1}^{M} l(x, ei)$$

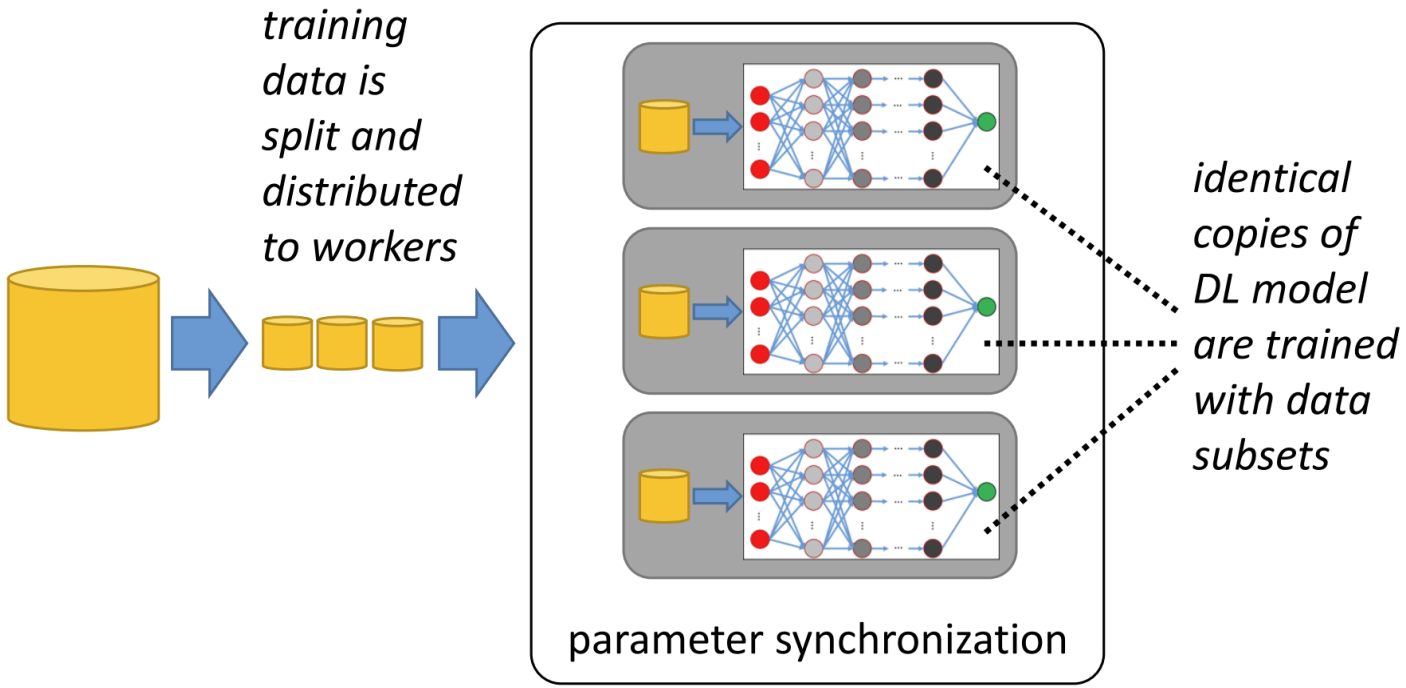


Distributed Machine Learning: A Brief Overview By Dan Alistarh, IST Austria.

Distributed ML: Parallelization

- Parallelization methods in DL: data, model and *pipeline*. Hybrid as well.

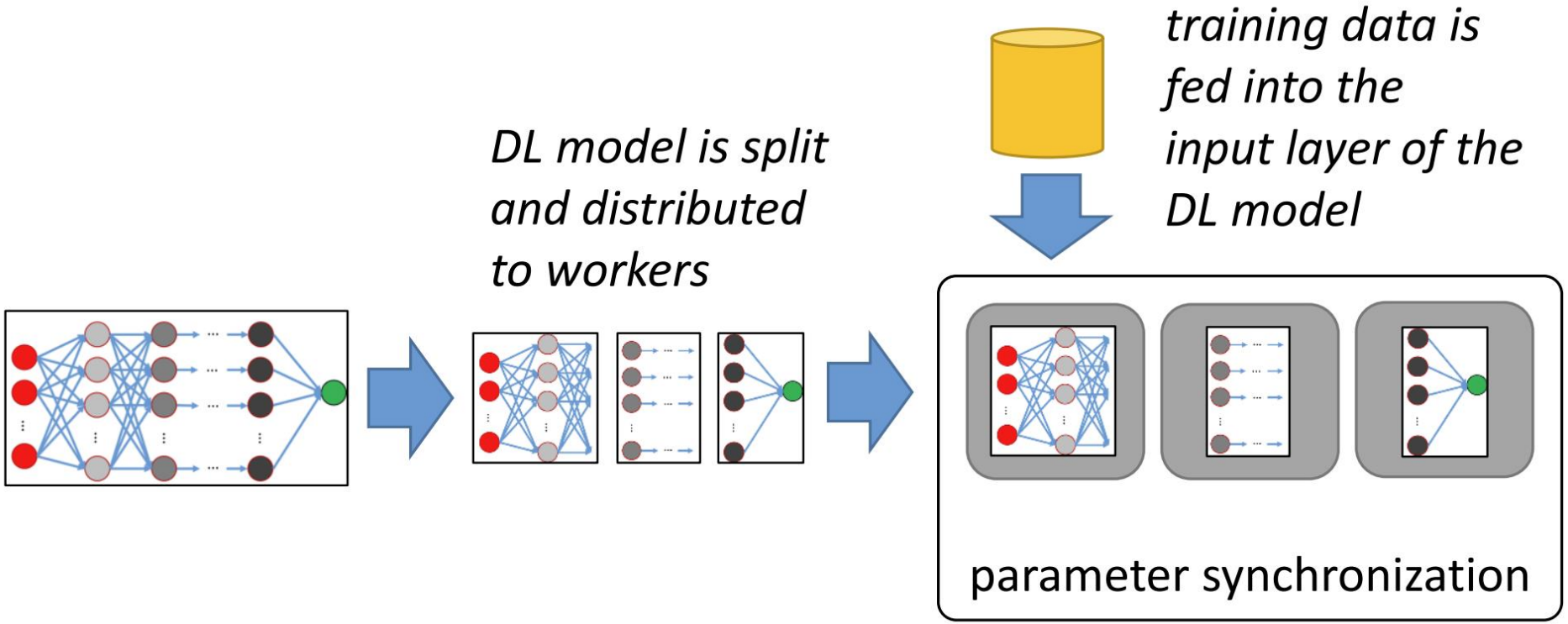
- In data parallelism, a number of machines loads an identical copy of a DL model.
- Training data split into non-overlapping chunks and fed into model replicas of workers for training.
- Each worker performs training on its training data, which leads to updates of model parameters.
- Hence, model parameters between workers need to be synchronized.



Mayer, Ruben, and Hans-Arno Jacobsen. "Scalable deep learning on distributed infrastructures: Challenges, techniques, and tools." *ACM Computing Surveys (CSUR)* 53.1 (2020): 1-37.

Distributed ML: Parallelization

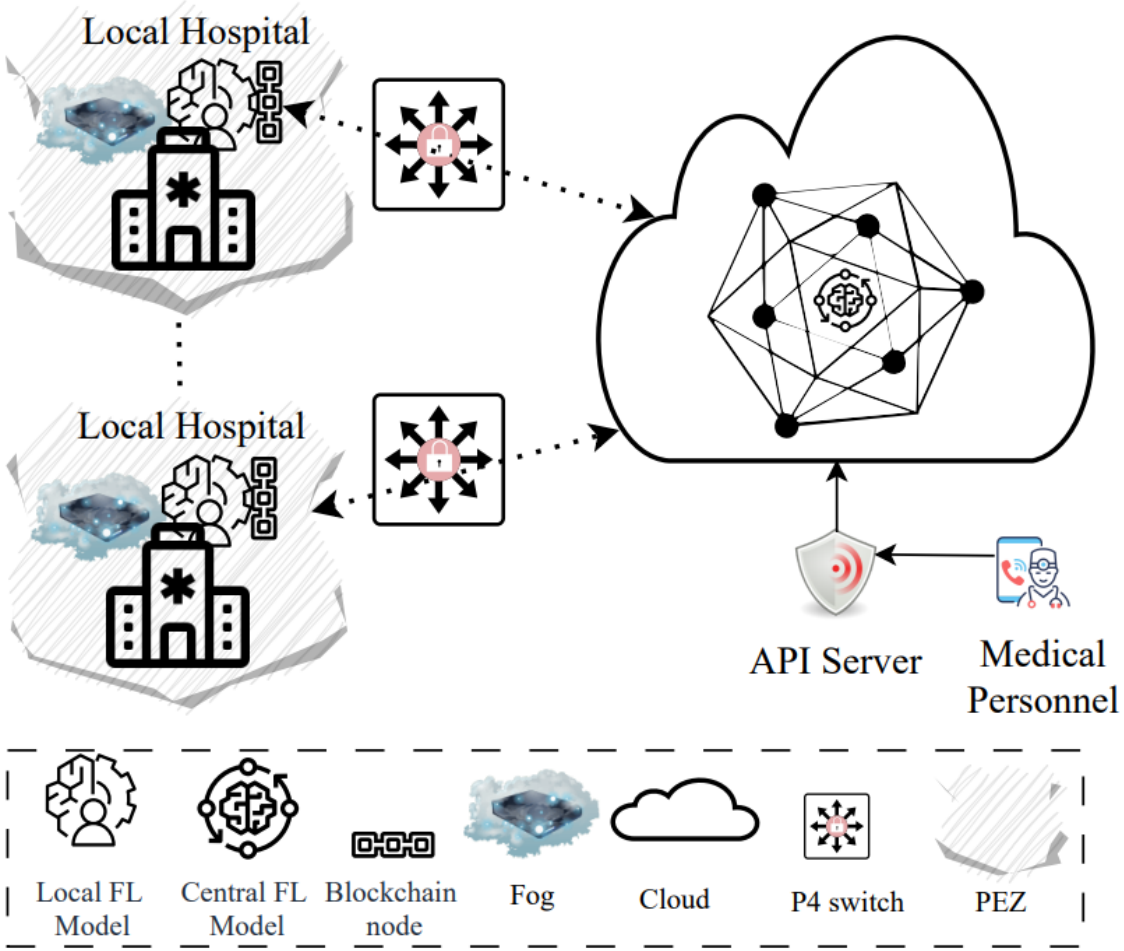
- Model parallelism: DL model is split, and each worker loads a different part of DL model for training.
- Worker(s) hold input layer of DL model are fed with training data.
- In forward pass, they compute their output signal which is propagated to workers that hold next layer of DL model.
- In backpropagation pass, gradients are computed starting at workers that hold output layer of DL model, propagating to workers that hold input layers of DL model.



Mayer, Ruben, and Hans-Arno Jacobsen. "Scalable deep learning on distributed infrastructures: Challenges, techniques, and tools." *ACM Computing Surveys (CSUR)* 53.1 (2020): 1-37.

Example

- Goal: To enhance the FL-infrastructural security and model-focused traceability challenges for medical analysis.



https://web-backend.simula.no/sites/default/files/2025-07/ISPDC2025-FL2.pdf

TensorFlow

TensorFlow and Others

	TensorFlow	PyTorch	Keras
API Level	Both (High and Low)	Low	High
Architecture	Not easy to use	Complex, less readable	Simple, concise, readable
Datasets	Large datasets, high-performance	Large datasets, high-performance	Smaller datasets
Debugging	Difficult to conduct debugging	Good debugging capabilities	Simple network, so debugging is not often needed
Pretrained models?	Yes	Yes	Yes
Popularity	Second most popular of the three	Third most popular of the three	Most popular of the three
Speed	Fast, high-performance	Fast, high-performance	Slow, low performance
Written In	C++, CUDA, Python	Lua	Python

https://www.datacamp.com/tutorial/pytorch-vs-tensorflow-vs-keras

What is TensorFlow?

- TensorFlow (tf) offers tools, libraries and an open-source platform for ML.
  - Keras python based deep learning framework (high-level API of tf).
- Perform numerical computation using data flow graphs.

- Developed by Google Brain Team to conduct ML research
- TensorFlow provides Python and C++ APIs.

- To install:
  - $ pip install tensorflow
  - $ pip install tensorflow-cpu (CPU-only package)
- Sample Code:

```
import tensorflow as tf
tf.add(1, 2).numpy()
```

What is TensorFlow (TF)?

- An open-source numerical computing and DL framework developed by Google.
- TF is conceptually similar to NumPy, but with:
  - GPU and TPU acceleration
  - Automatic differentiation
  - Distributed training support
- TF represents numerical computations as a computational graph:
  - Nodes --> mathematical operations
  - Edges --> tensors (multi-dimensional arrays) flowing between operations
  - Graphs define dependencies and execution order
- Computation graphs can be exported to portable formats (ONNX) enabling:
  - Train in one environment → Deploy in another

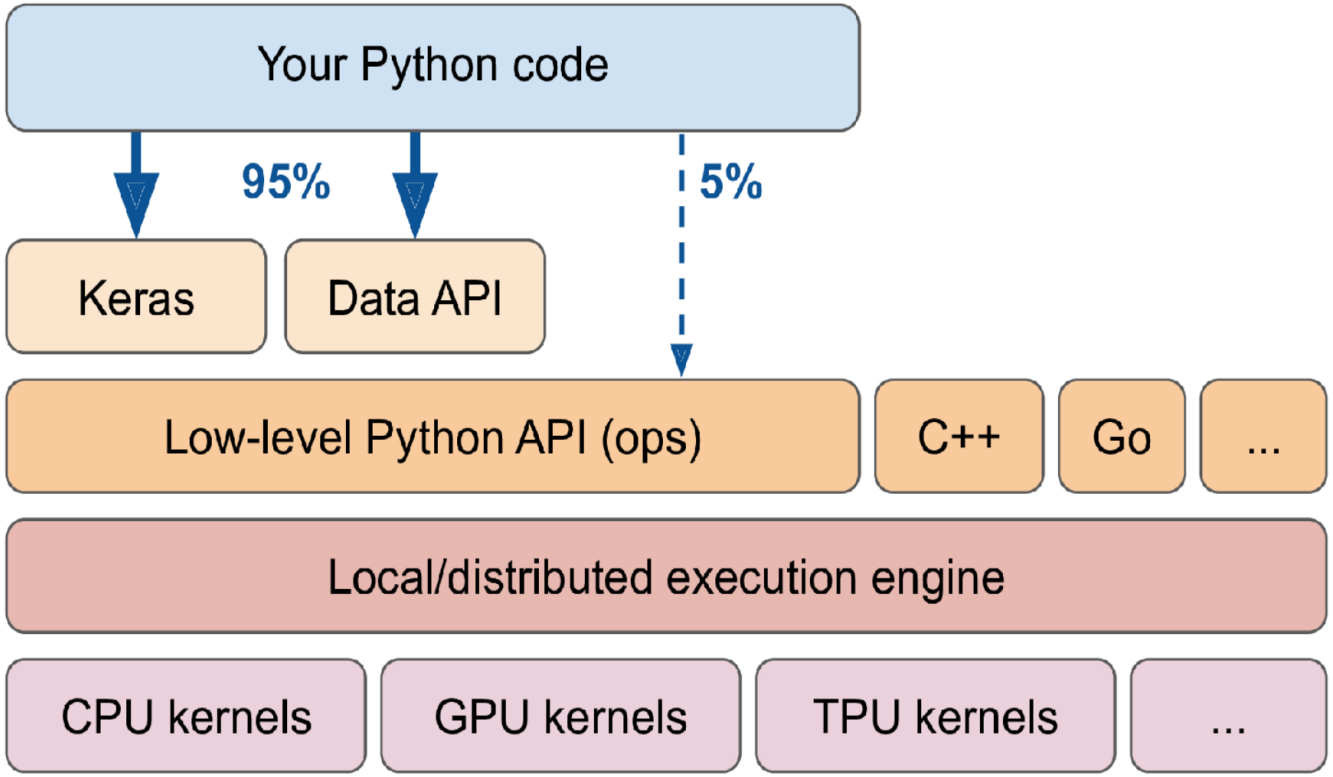
- **tf.keras** – High-level API for building and training neural networks
- **tf.data** – Efficient data input pipelines
- **tf.io** – Data serialization and file I/O utilities
- **tf.image** – Image processing operations
- **tf.signal** – Signal processing functions

What is TensorFlow?

- TensorFlow's API revolves around tensors, which flow between operations
  - A tensor is conceptually similar to a NumPy ndarray: (a multi-dimensional array)
  - Represent a scalar (0-dimensional tensor)
  - Supports various data types (float, int, etc.)
- Interoperability with NumPy
  - Tensors can be created from and converted back to NumPy arrays
  - TF operations can be applied to NumPy arrays
  - NumPy operations can be applied to tensors (when compatible)
- Tensors are essential when implementing:
  - Custom loss functions
  - Custom evaluation metrics
  - Custom layers
  - Advanced model components

TensorFlow's Architecture

- TensorFlow's execution engine manages:
- Automatic optimization of computation graphs
- Parallel execution
- GPU/TPU acceleration
- Distributed execution across multiple devices and machines



TensorFlow's Python API

- At lowest level, each TensorFlow operation is implemented using C++ code.
- Many operations have multiple implementations called kernels.
- Each dedicated to a specific device (CPUs/GPUs/TPUs).
- GPUs can speed up computations by splitting them into many smaller chunks and running them in parallel.
- TPUs are custom ASIC chips built specifically for DL operations.

`tf.keras` `tf.estimator`	High-level Deep Learning APIs

`tf.nn`
`tf.losses`
`tf.metrics`
`tf.optimizers`
`tf.train`
`tf.initializers`
	Low-level Deep Learning APIs

`tf.GradientTape` `tf.gradients()`	Autodiff

`tf.data`
`tf.feature_column`
`tf.audio`
`tf.image`
`tf.io`
`tf.queue`
	I/O and preprocessing

`tf.summary`	Visualization with TensorBoard

`tf.distribute`
`tf.saved_model`
`tf.autograph`
`tf.graph_util`
`tf.lite`
`tf.quantization`
`tf.tpu`
`tf.xla`
	Deployment and optimization

`tf.lookup`
`tf.nest`
`tf.ragged`
`tf.sets`
`tf.sparse`
`tf.strings`
	Special data structures

`tf.math`
`tf.linalg`
`tf.signal`
`tf.random`
`tf.bitwise`
	Mathematics, including linear algebra and signal processing

`tf.compat` `tf.config` *& more*	Miscellaneous

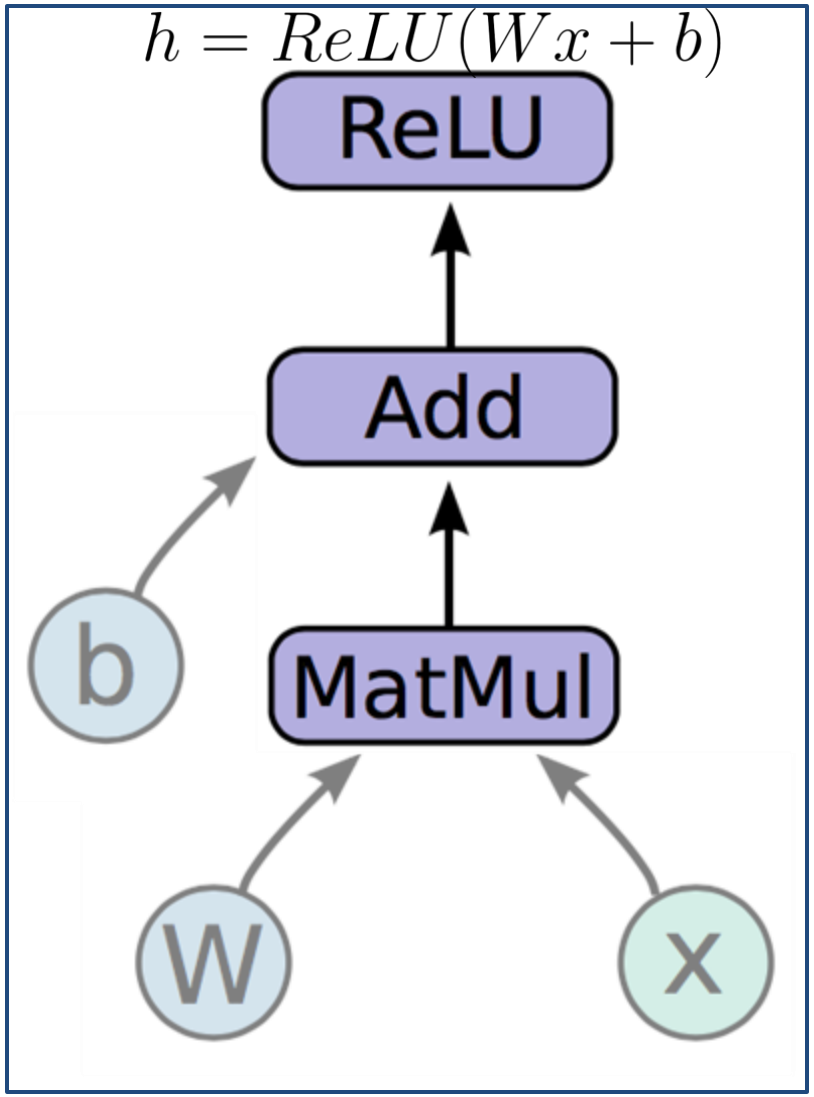
How it works?

- MatMul: Multiply two matrices
- Add: Add element wise
- ReLU: Activate with elementwise rectified linear function

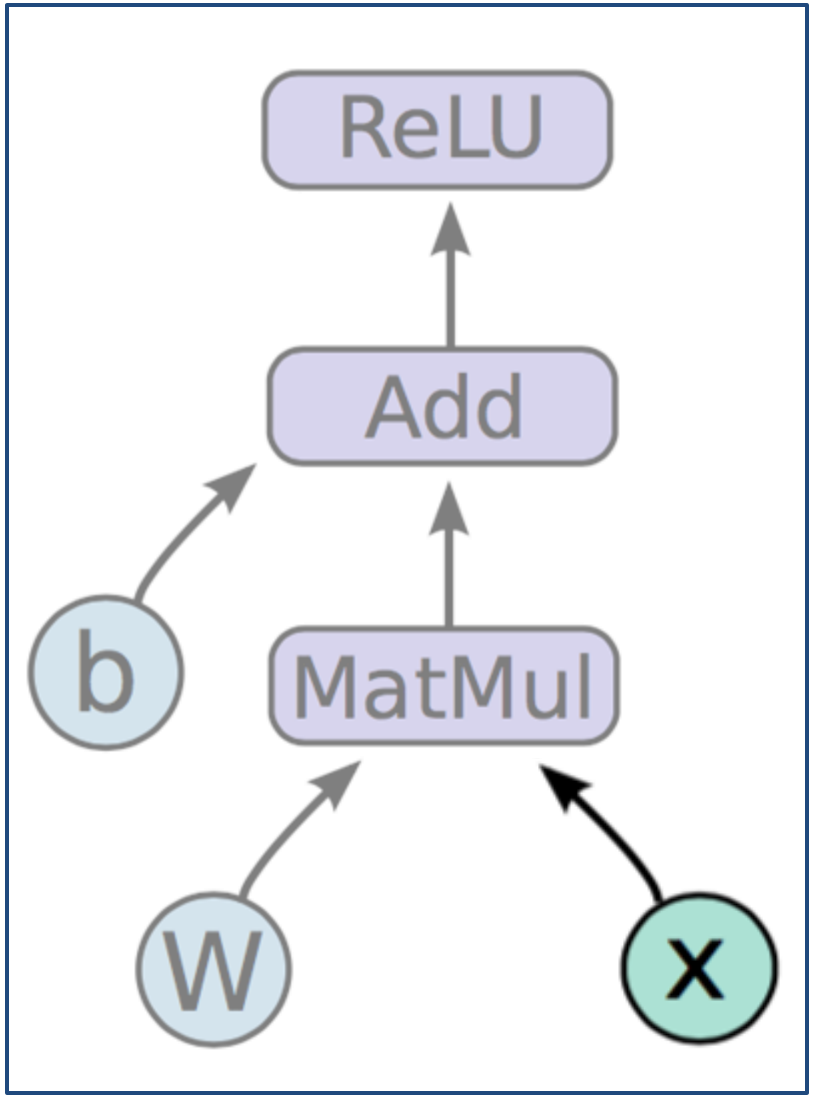
$$ReLu(x) = \begin{cases} 0, & x <= 0 \\ x, & x > 0 \end{cases}$$

- tf.Variable represents mutable state.
- Stores a tensor value that can be updated (e.g., during training).
- Provides its current stored value.

- Persists across forward passes.
- Model parameters (weights, biases) are stored as variables.
- Values are updated via optimizers and retained between steps.
- All trainable weights inside layers are tf.Variable objects and retain state across batches and epochs.



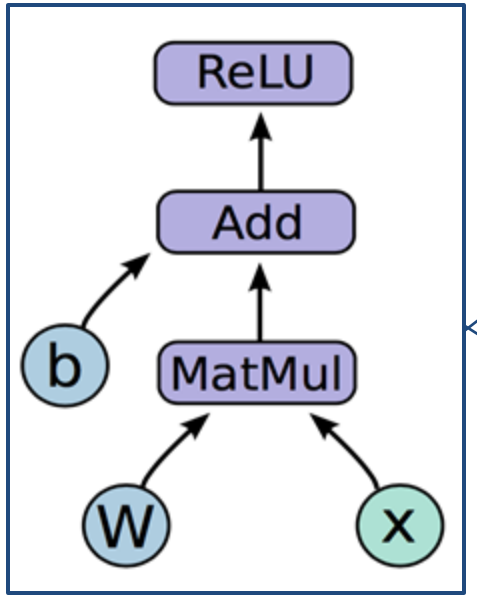
How it works?



$$h = ReLU(Wx + b)$$

- Placeholders are nodes whose value is fed in at execution time (inputs, labels, …).
- Code for:

$$h = ReLU(Wx + b)$$

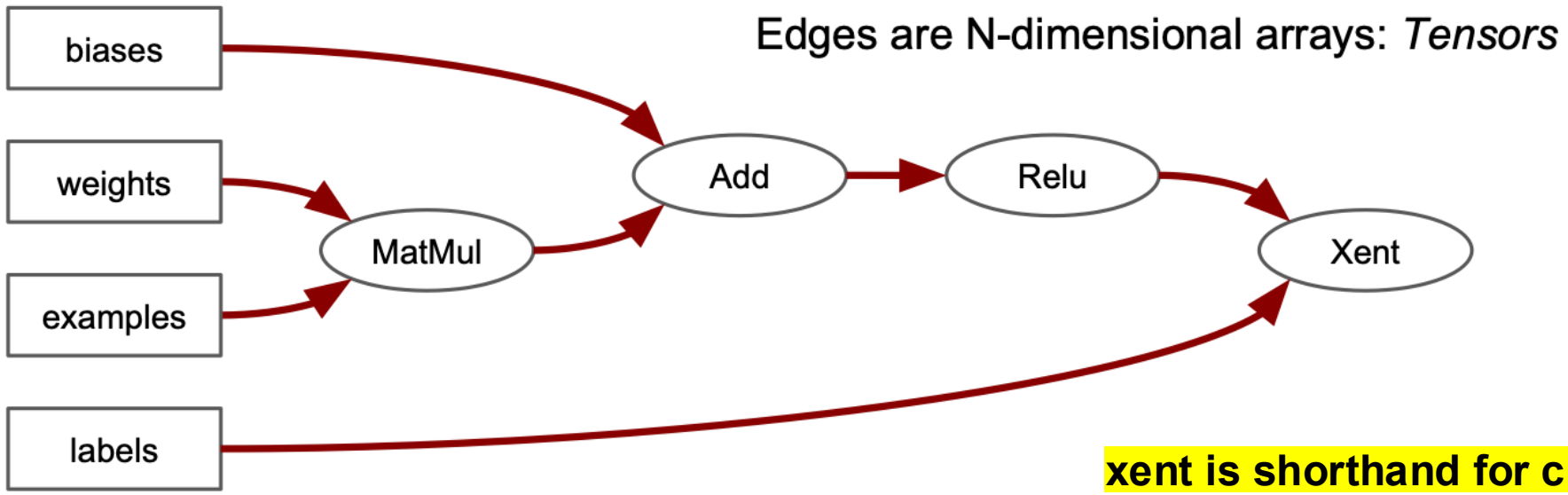


CPU

- Deploy graph with a session

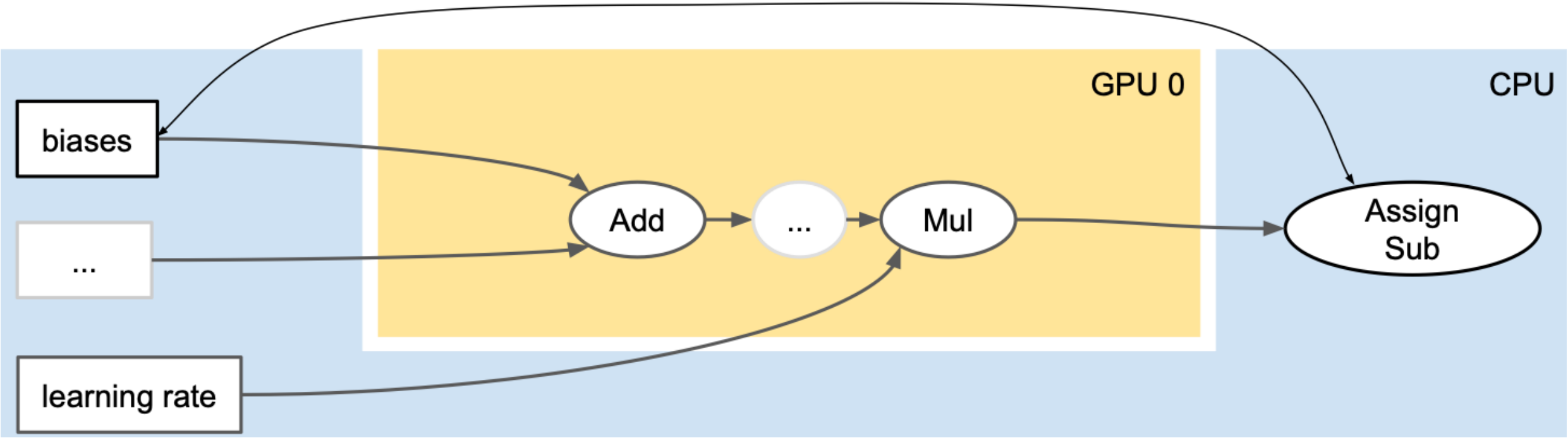
GPU

TensorFlow's Computation: Dataflow Graph

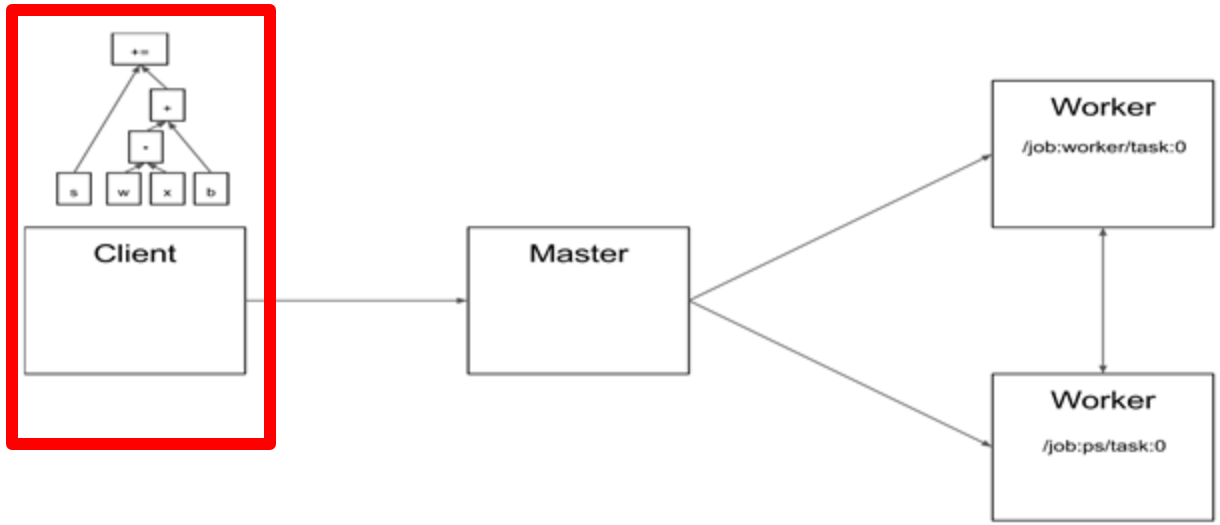


xent is shorthand for cross-entropy loss

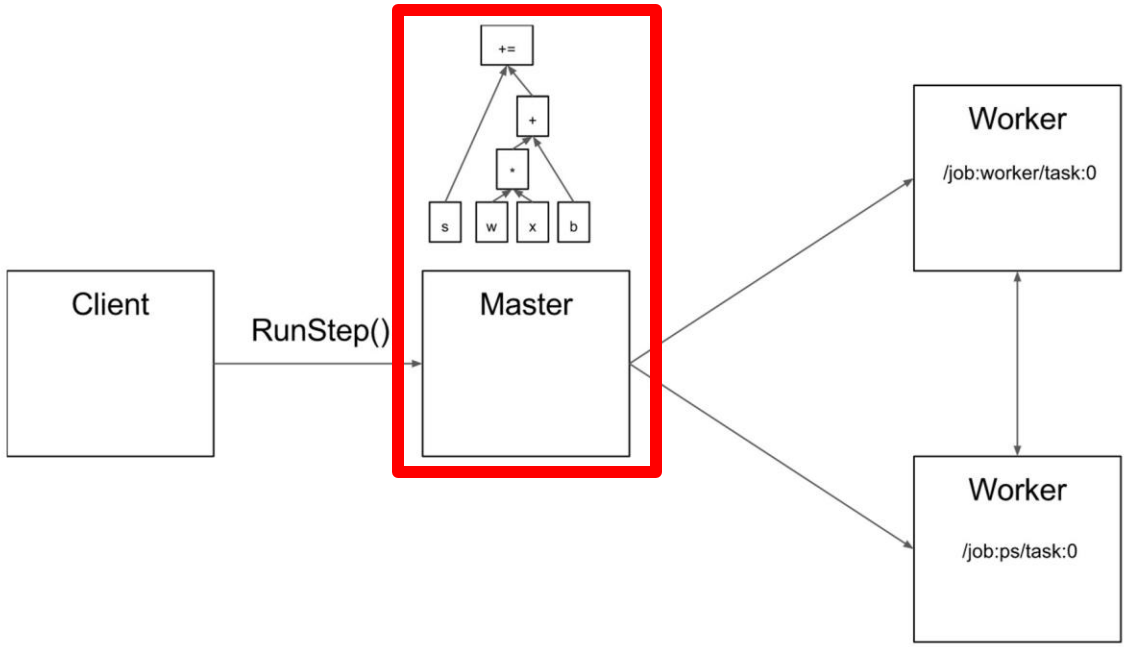
TensorFlow's Computation: Dataflow Graph Distribution



TensorFlow's Execution Framework



- Client



- Master

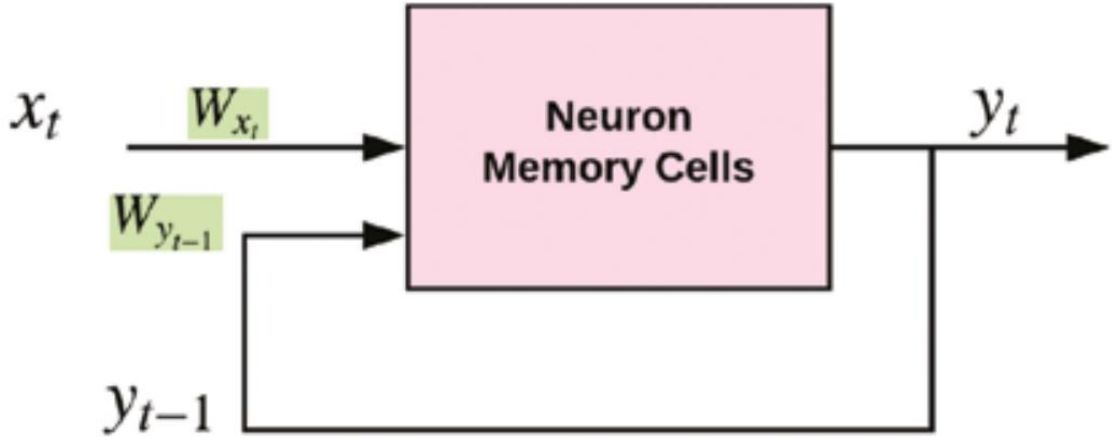
https://www.tensorflow.org/extend/architecture

RNN: Recurrent Neural Networks

Recurrent Neural Networks (RNNs)

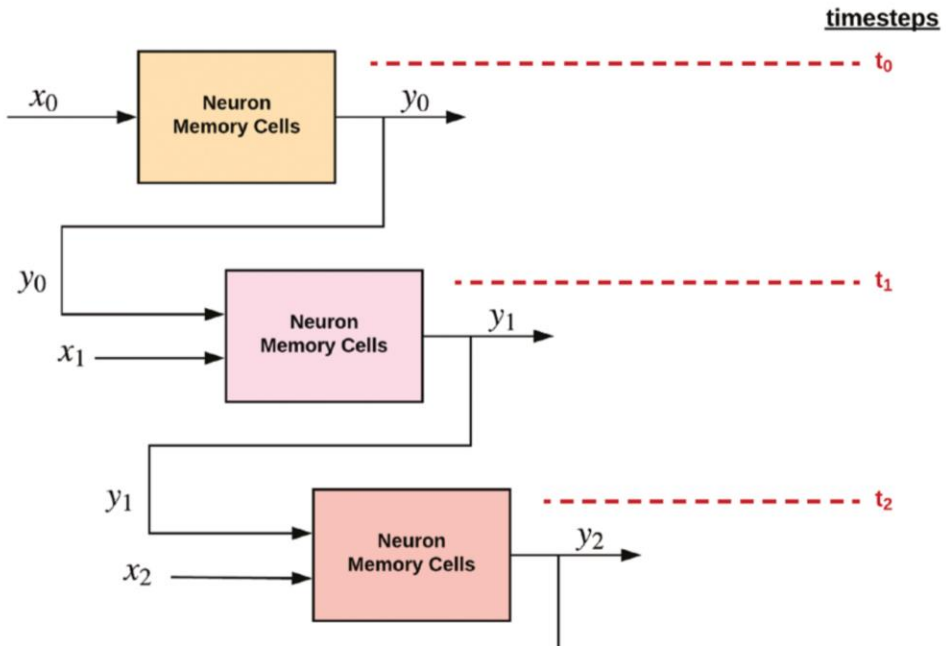
- RNNs developed to solve learning problems (time series or sequential tasks) where information about past (i.e., past instants/events) is directly linked to making future predictions.

- Recurrent Neuron: maintains a memory or a state from past computations.
  - Data is looped back into same neuron at every new time instant.
  - It takes input as output of previous instant y_{t-1} in addition to its current input at instant x_t.
  - The recurrent neuron has two input weights, W_{xt} and W_{yt-1}



Recurrent Neural Networks (RNNs)

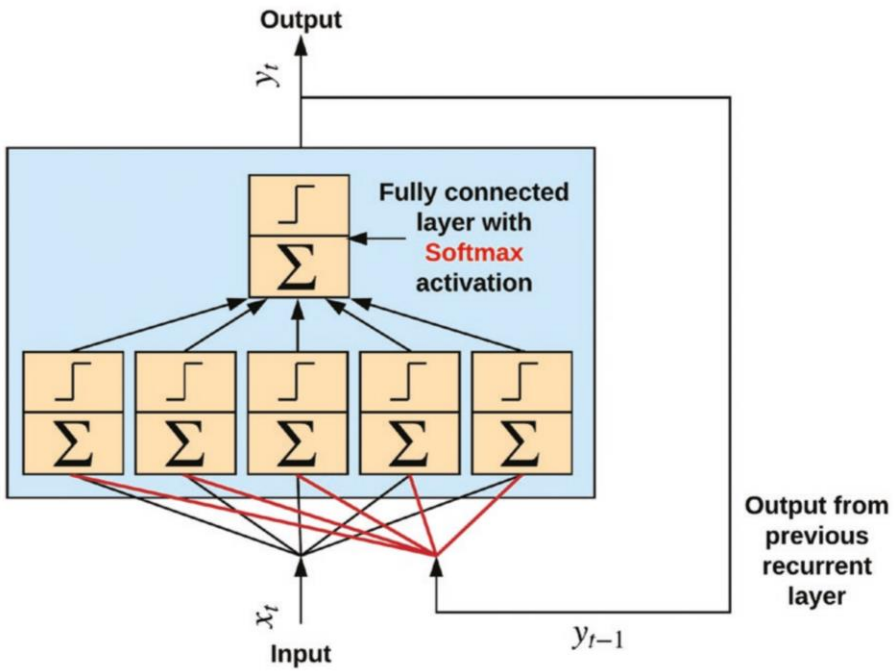
- Recurrent Computational Graph: RNN formalized as an unfolded computational graph.
- An unfolded computational graph shows information flow through recurrent layer at every time instant in the sequence.



A sequence of five-time steps: We will unfold recurrent neuron five-times across number of instants.

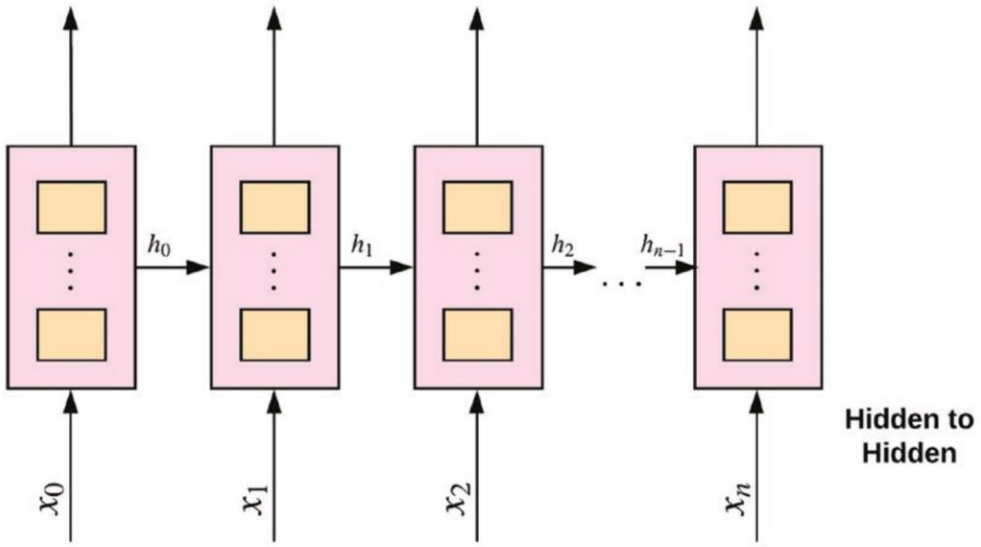
Recurrent Neural Networks (RNNs)

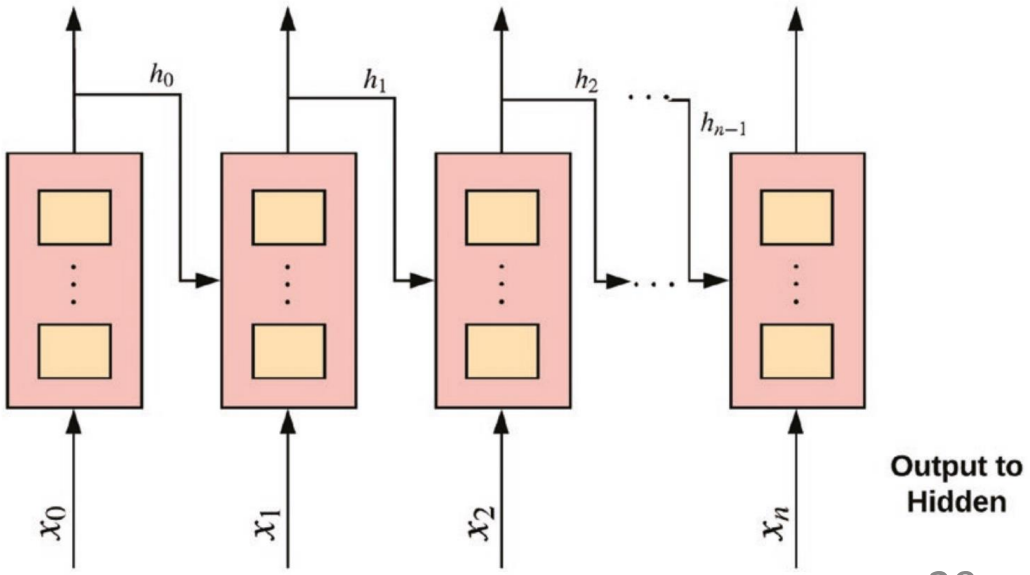
- Computations within a recurrent layer:
  - Each neuron in a recurrent layer (RL) receives as input, output of previous layer and its current input.
  - Neurons each have two weight vectors.
  - Neurons perform an affine transformation of inputs and pass it through a non-linear activation function (tanh).
  - Within RL, output of neurons is moved to a dense or fully connected layer with a softmax activation function as output of class probabilities.



Recurrent Connection Schemes

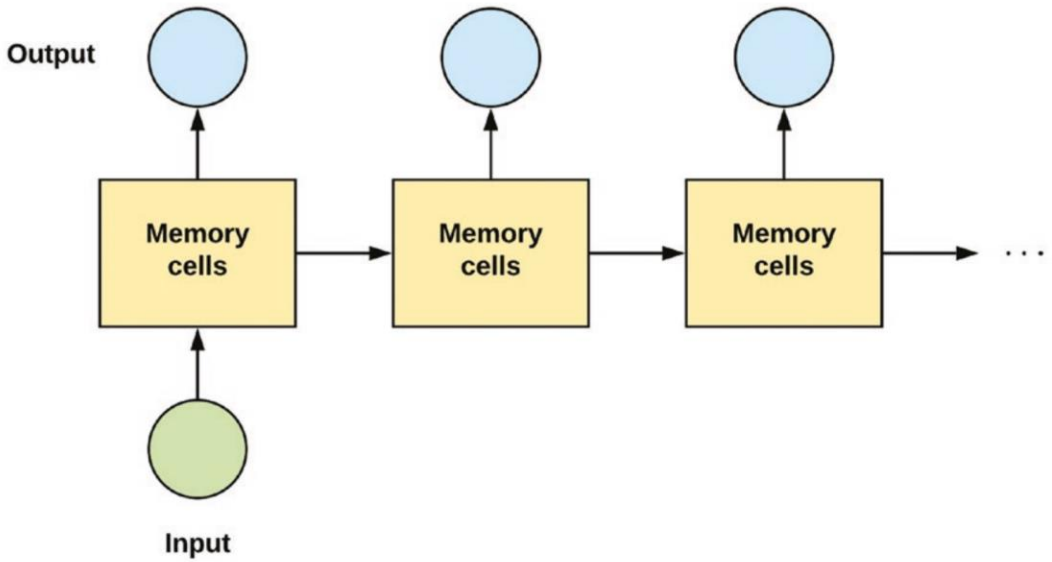
- Two main schemes for forming recurrent connections from one recurrent layer to another:
  - Recurrent connections between hidden units.
    - Better captures high-dimensional features about past.
  - Recurrent connections between output of previous layer and hidden unit.
    - Easy to compute and parallelizable.



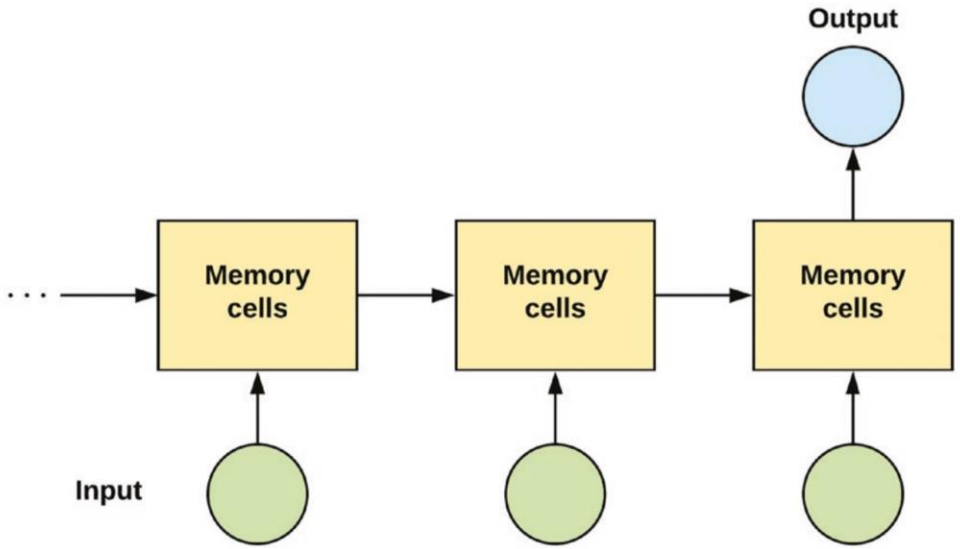


RNN for Sequence Problems

- An input to a sequence of output: when an image is passed as an input to network, and output is a sequence of words (Image captioning problem)



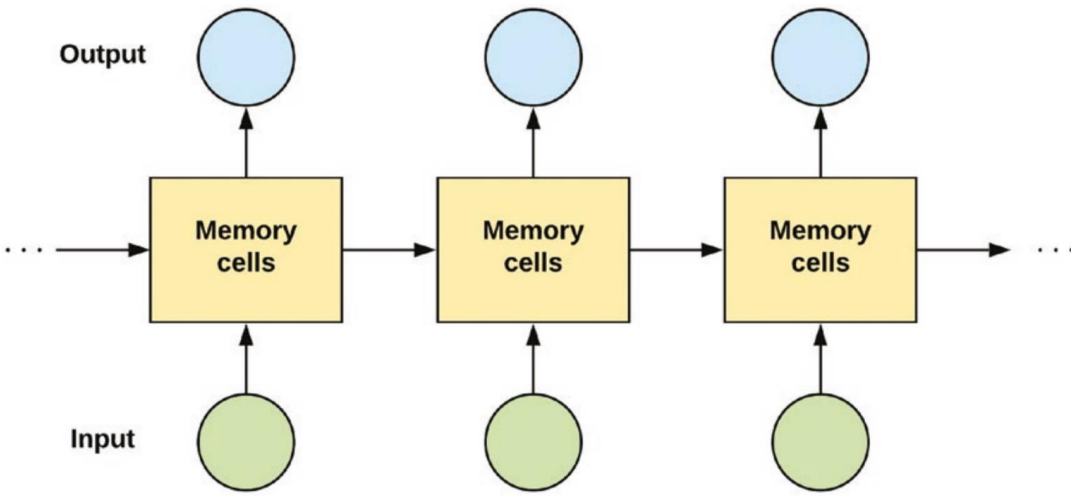
- A sequence of inputs to an output: Pass a sequence of words as input to network, and output is a class indicating either a positive or negative review or sentiment (Sentiment Analysis)

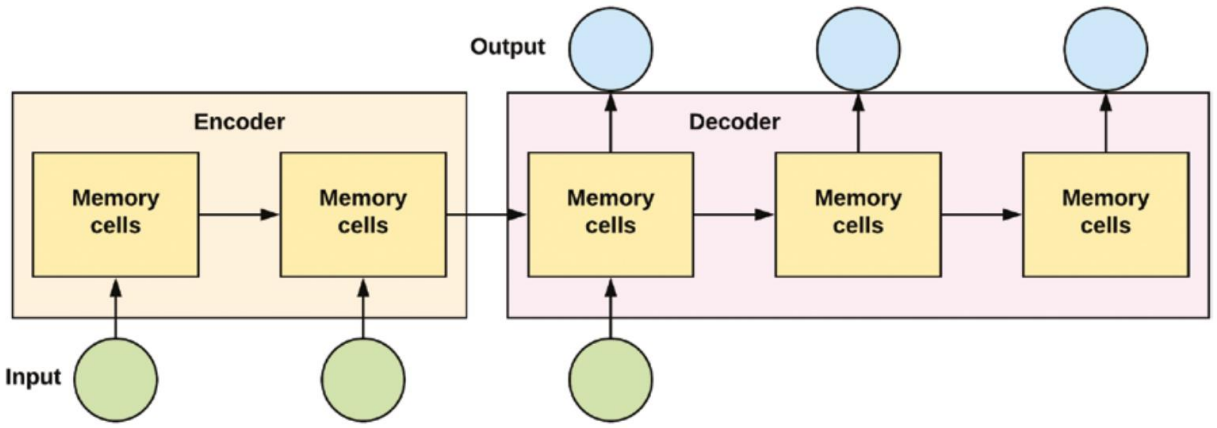


RNN for Sequence Problems

- Synced sequence input to output: need to label each video frame (Video Classification).

- Encoder-decoder/sequence-to-sequence architecture: A sequence of words in a language as input, and we want a sequence of words as output in another language (Machine translation and Speech recognition)





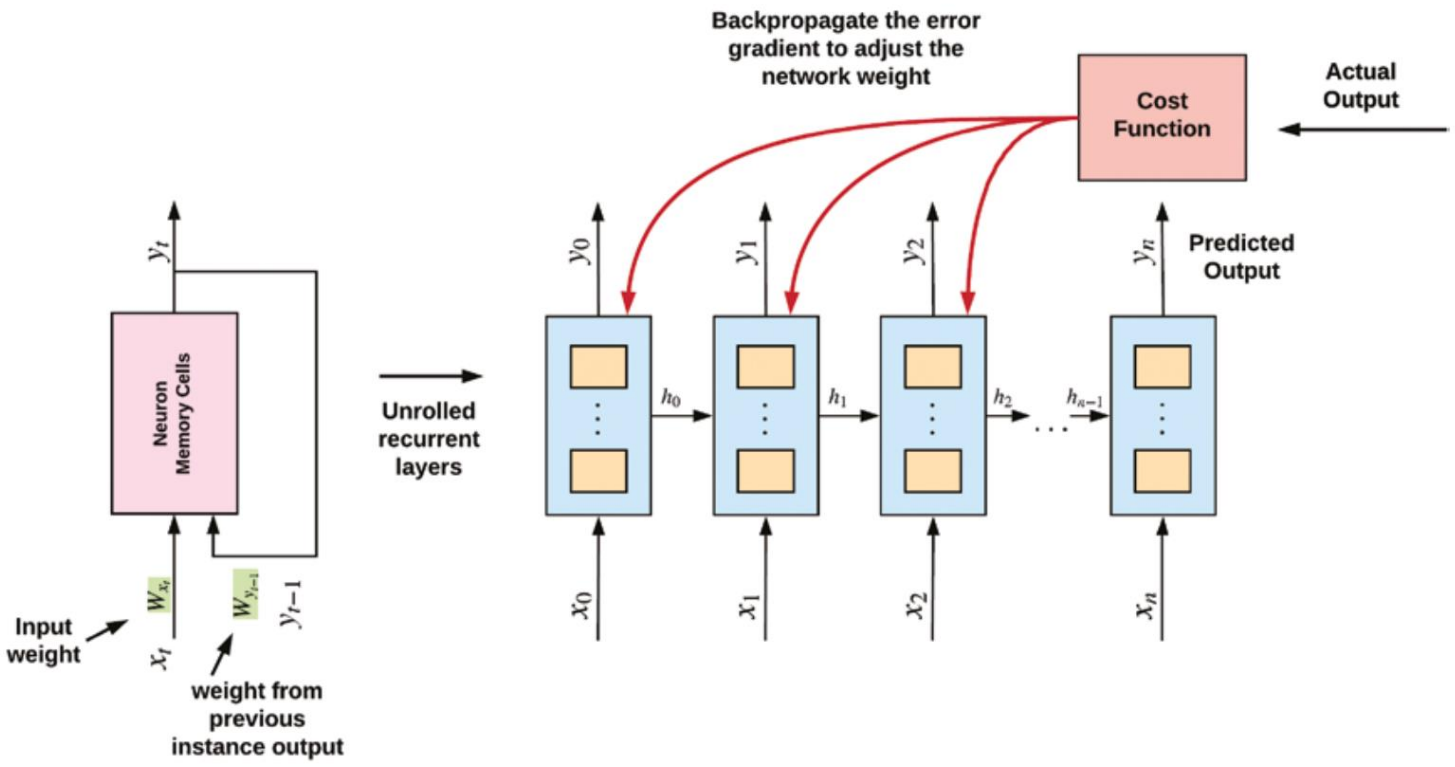
Recurrent Network Training

- RNN is trained by backpropagation through time (BPTT).
  - Because standard backpropagation cannot work in loop or recurrent structure.
    - Training a network using backpropagation involves calculating error gradient, moving backward from output layer through hidden layers of network and adjusting network weights.
  - However, this operation cannot work in recurrent neuron because we have just one neural cell with recurrent connections to itself.
    - Note: Deep RNN is the way to stack multiple layers of cells.
      - Each time step t (called a frame).

Recurrent Network Training

BPTT:

1. Unroll recurrent neuron across time instants
2. Apply backpropagation to unrolled neurons at each time layer same way it is done for a traditional feedforward NN.



Recurrent Network Training

- Challenge in training RNNs is the vanishing and exploding gradient problem.
  - During backpropagation through time (BPTT), gradients are propagated across many time steps.
  - For long sequences, gradients may:
    - Vanish (shrink toward zero), making it difficult to learn long-term dependencies.
    - Explode (grow excessively large), leading to unstable training.
- Standard RNNs struggle to capture long-range temporal dependencies.

- Mitigation Techniques: Several techniques:
  - Gradient clipping – prevents exploding gradients by capping their magnitude.
  - Batch normalization – stabilizes training dynamics.
  - ReLU activations – can alleviate vanishing gradients compared to sigmoid/tanh

- Limitations of vanilla RNNs led to the development of gated memory architectures such as Long Short-Term Memory (LSTM)

Example

```
tf.keras.layers.SimpleRNN(
    units,
    activation='tanh',
    use_bias=True,
    kernel_initializer='glorot_uniform',
    recurrent_initializer='orthogonal',
    bias_initializer='zeros',
    kernel_regularizer=None,
    recurrent_regularizer=None,
    bias_regularizer=None,
    activity_regularizer=None,
    kernel_constraint=None,
    recurrent_constraint=None,
    bias_constraint=None,
    dropout=0.0,
    recurrent_dropout=0.0,
    return_sequences=False,
    return_state=False,
    go_backwards=False,
    stateful=False,
    unroll=False,
    seed=None,
    **kwargs
)
```

Call arguments	
`sequence`	A 3D tensor, with shape `[batch, timesteps, feature]`.
`mask`	Binary tensor of shape `[batch, timesteps]` indicating whether a given timestep should be masked. An individual `True` entry indicates that the corresponding timestep should be utilized, while a `False` entry indicates that the corresponding timestep should be ignored.
`training`	Python boolean indicating whether the layer should behave in training mode or in inference mode. This argument is passed to the cell when calling it. This is only relevant if `dropout` or `recurrent_dropout` is used.
`initial_state`	List of initial state tensors to be passed to the first call of the cell.

Args	
`units`	Positive integer, dimensionality of the output space.
`activation`	Activation function to use. Default: hyperbolic tangent (`tanh`). If you pass None, no activation is applied (ie. "linear" activation: `a(x) = x`).
`use_bias`	Boolean, (default `True`), whether the layer uses a bias vector.
`kernel_initializer`	Initializer for the `kernel` weights matrix, used for the linear transformation of the inputs. Default: `"glorot_uniform"`.
`recurrent_initializer`	Initializer for the `recurrent_kernel` weights matrix, used for the linear transformation of the recurrent state. Default: `"orthogonal"`.
`bias_initializer`	Initializer for the bias vector. Default: `"zeros"`.
`kernel_regularizer`	Regularizer function applied to the `kernel` weights matrix. Default: `None`.
`recurrent_regularizer`	Regularizer function applied to the `recurrent_kernel` weights matrix. Default: `None`.
`bias_regularizer`	Regularizer function applied to the bias vector. Default: `None`.
`activity_regularizer`	Regularizer function applied to the output of the layer (its "activation"). Default: `None`.
`kernel_constraint`	Constraint function applied to the `kernel` weights matrix. Default: `None`.
`recurrent_constraint`	Constraint function applied to the `recurrent_kernel` weights matrix. Default: `None`.
`bias_constraint`	Constraint function applied to the bias vector. Default: `None`.
`dropout`	Float between 0 and 1. Fraction of the units to drop for the linear transformation of the inputs. Default: 0.
`recurrent_dropout`	Float between 0 and 1. Fraction of the units to drop for the linear transformation of the recurrent state. Default: 0.
`return_sequences`	Boolean. Whether to return the last output in the output sequence, or the full sequence. Default: `False`.
`return_state`	Boolean. Whether to return the last state in addition to the output. Default: `False`.
`go_backwards`	Boolean (default: `False`). If `True`, process the input sequence backwards and return the reversed sequence.
`stateful`	Boolean (default: `False`). If `True`, the last state for each sample at index i in a batch will be used as initial state for the sample of index i in the following batch.
`unroll`	Boolean (default: `False`). If `True`, the network will be unrolled, else a symbolic loop will be used. Unrolling can speed-up a RNN, although it tends to be more memory-intensive. Unrolling is only suitable for short sequences.

Long Short-Term Memory (LSTM)

Long Short-Term Memory (or LSTM)

- LSTM is a RNN variant.
  - More complicated memory cell than RNN.
  - Proposed in 1997 by Sepp Hochreiter and Jürgen Schmidhuber*.
  - Contain gates to control information flow within recurrent cell.
  - Efficient for capturing long-term dependencies across many time instants.
  - Can be used for faster training, long-term data dependency detection.

- Components of LSTM: Memory cell, Input gate, Forget gate, Output gate.

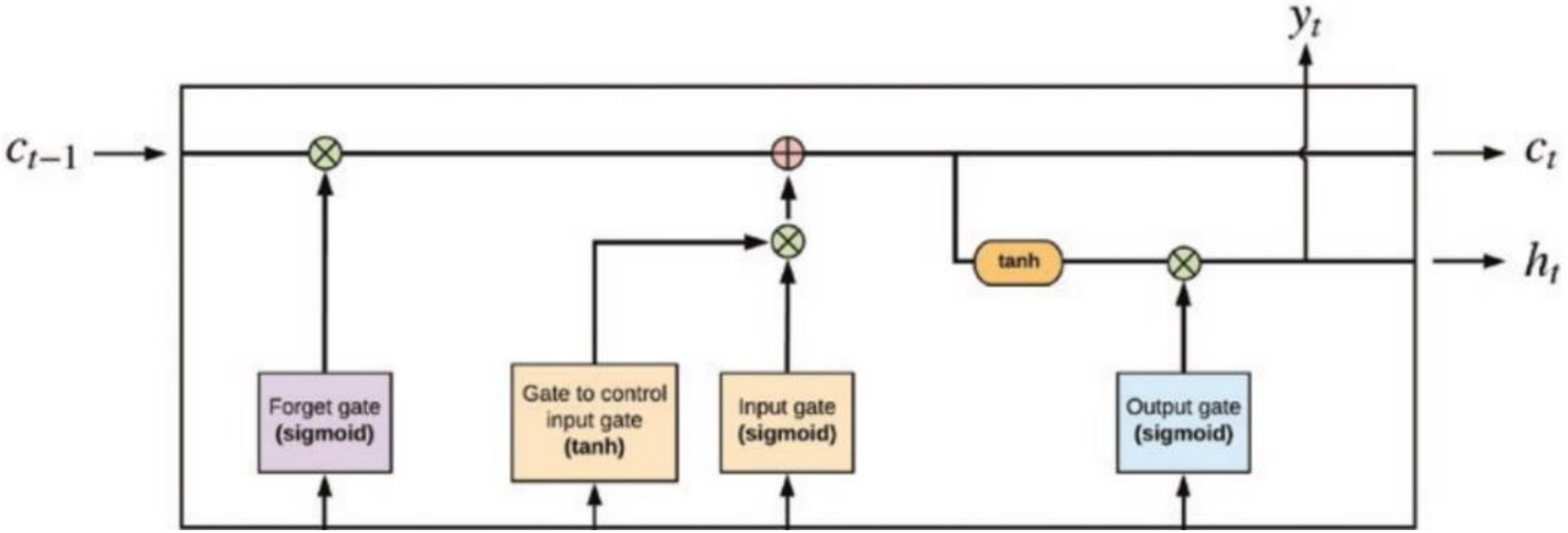
- Applications: Image captioning, stock market prediction, machine translation, and text classification (for sequence processing).

*Sepp Hochreiter and Jürgen Schmidhuber, "Long Short-Term Memory," Neural Computation 9, no. 8 (1997): 1735–1780.

LSTM Components

- Extra components enable RNN to remember and store important events from distant past.

- Input gate: controls what information gets stored in long-term state/memory cell (c).
  - There is another gate that regulates information flowing into input gate.
  - This gate analyzes current input to LSTM cell, x_t and previous short-term state (h_t−1).



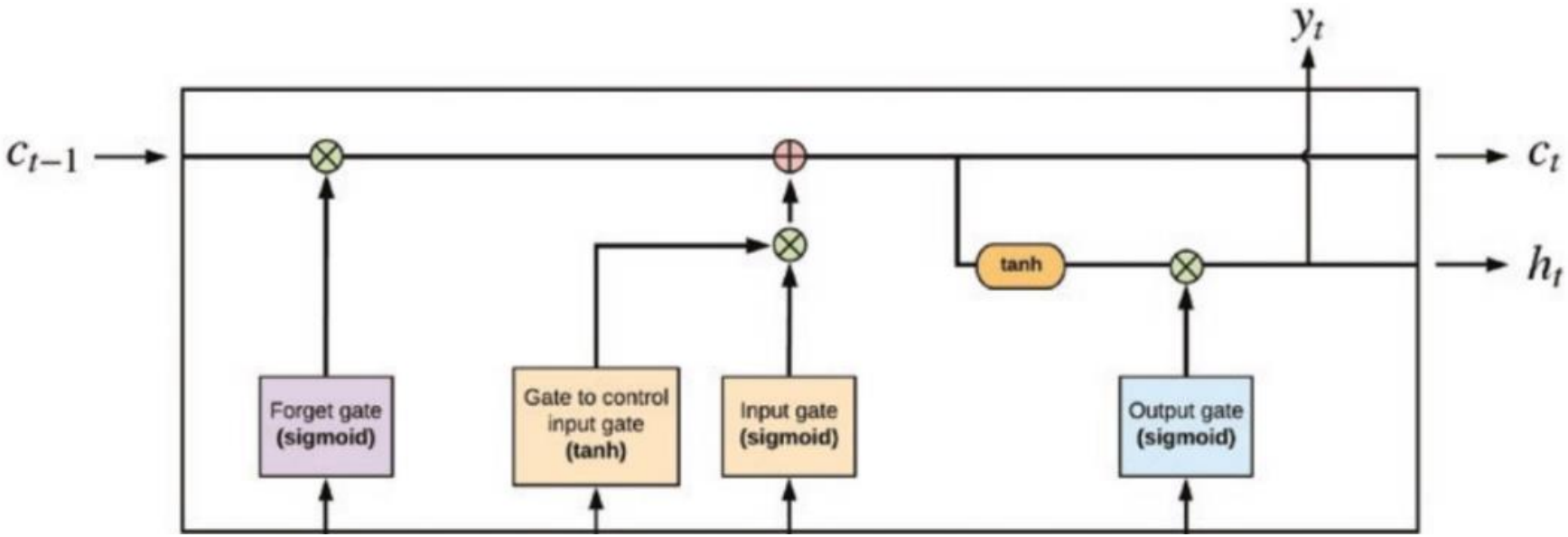
Notations: Previous cell state (c_t−1), Previous hidden state (h_t−1), Current input x_t.

LSTM Components

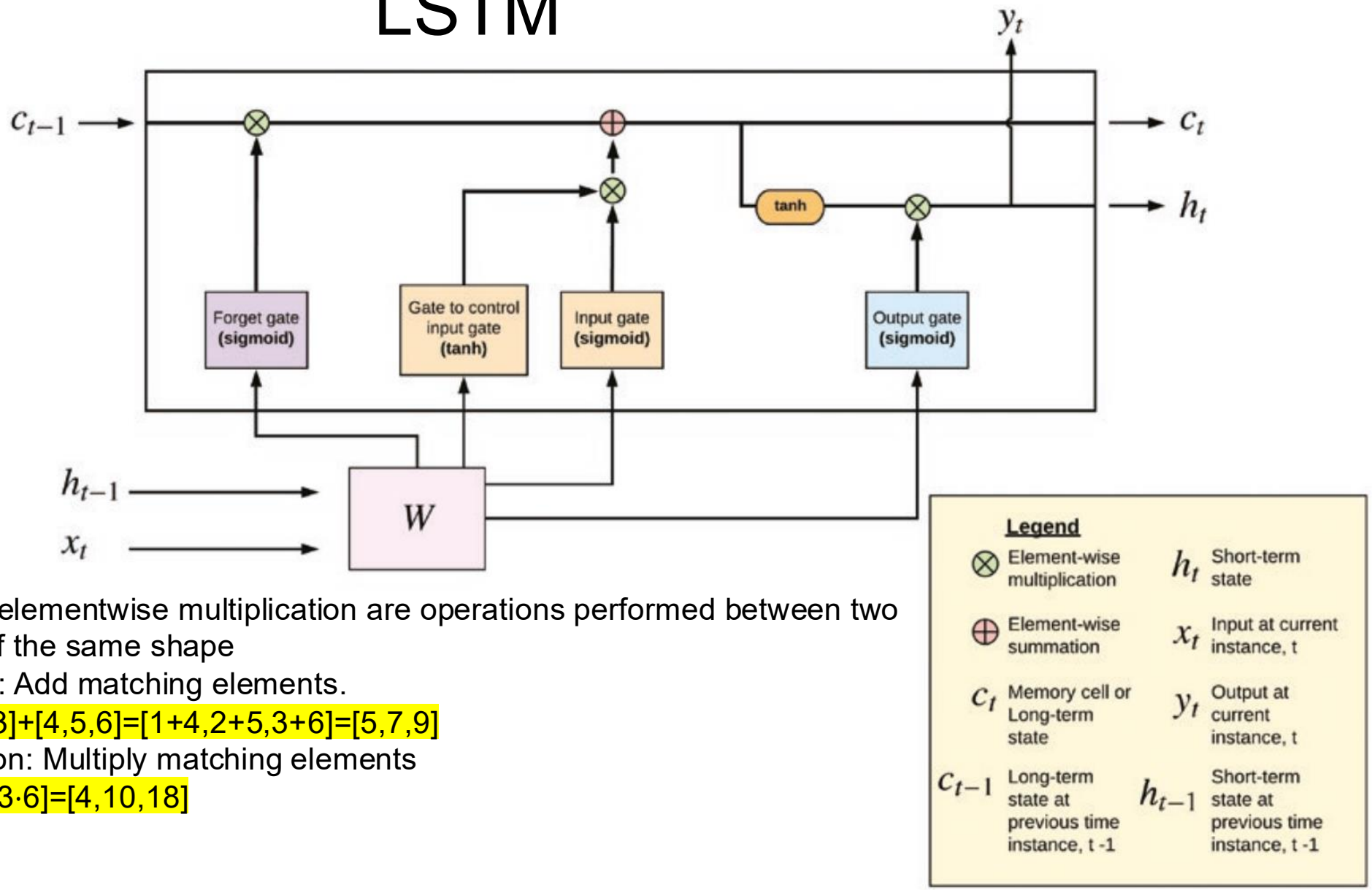
- Forget gate: Regulate how much of information in long-term state is persisted across time instants.
- Note: Components of LSTM cells are fully connected NNs.
  - Variants of RNs with memory cells are peephole connections and gated recurrent units.

- Output gate: controls how much information to output from cell at a time instant.
  - Control value of h_t (short-term state) and y_t (output at time t).

Notations:
- Previous cell state (c_t−1)
- Previous hidden state (h_t−1)
- Current input x_t.

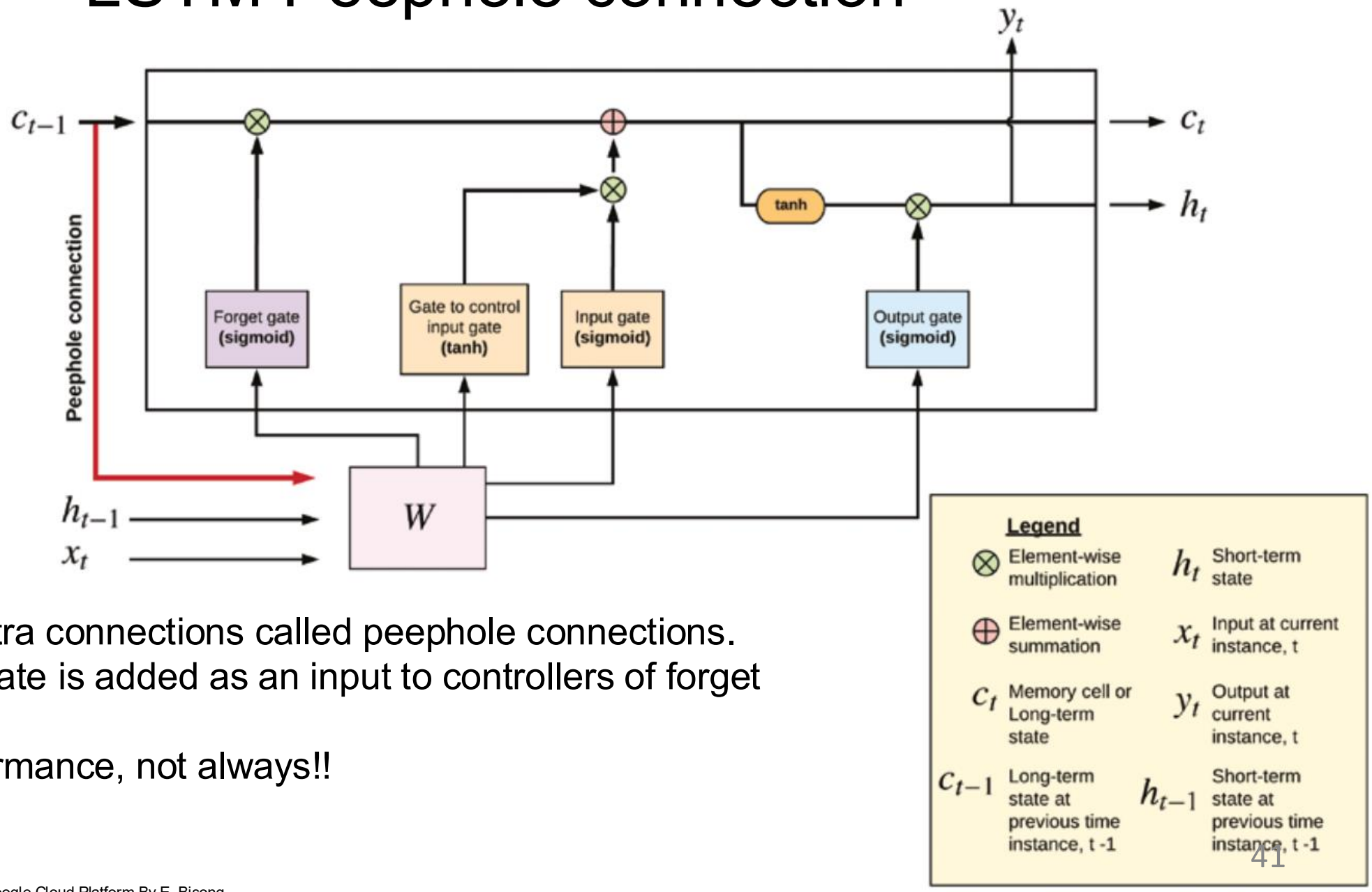


LSTM



- Elementwise summation and elementwise multiplication are operations performed between two arrays (vectors or matrices) of the same shape
  - Elementwise summation: Add matching elements.
    Example (vectors): [1,2,3]+[4,5,6]=[1+4,2+5,3+6]=[5,7,9]
  - Elementwise multiplication: Multiply matching elements
  - [1,2,3]⊙[4,5,6]=[1·4,2·5,3·6]=[4,10,18]

LSTM Peephole connection



- LSTM variant with extra connections called peephole connections.
- Previous long-term state is added as an input to controllers of forget gate and input gate.
- Often improves performance, not always!!

LSTM Code Snippet

LSTM cell can recognize an important input, store it in long-term state, preserve it for if it is needed and extract it whenever it is needed.

```
model = keras.models.Sequential([
keras.layers.LSTM(20, return_sequences=True,
input_shape=[None, 1]),
keras.layers.LSTM(20, return_sequences=True),
keras.layers.TimeDistributed(keras.layers.Dense(10)) ])
```

Using LSTM layer NOT SimpleRNN layer

```
model = keras.models.Sequential([
keras.layers.RNN(keras.layers.LSTMCell(20),
return_sequences=True,
input_shape=[None, 1]),
keras.layers.RNN(keras.layers.LSTMCell(20),
return_sequences=True),
keras.layers.TimeDistributed(keras.layers.Dense(10))
])
```

LSTMCell as an argument

- Two stacked LSTMs extract temporal features.
- A dense layer produces 10 outputs at each time step.

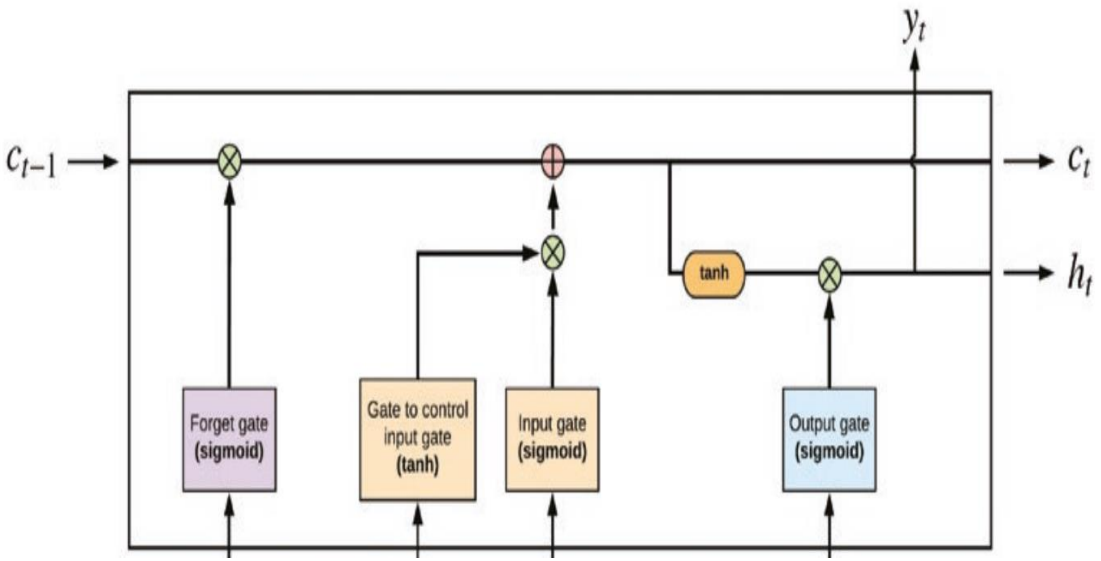
Gated Recurrent Unit (GRU)

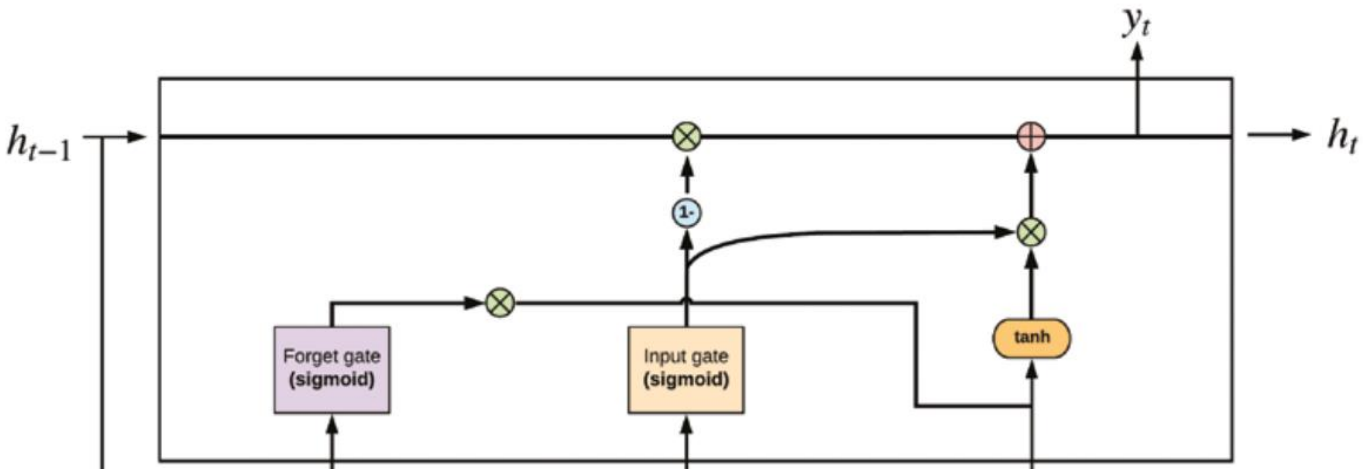
- Variants of LSTM cell exists such as GRU (Gated Recurrent Unit (GRU) cell*.
- LSTM and GRU cells are reasons behind success of RNNs.
- GRU cell is a simplified version of LSTM cell.
- Can handle much longer sequences than simple RNNs.

- GRUs combine forget and input gates to decide on what information should be committed to long-term memory (memory cell) and be left out.
    - Combines long and short-term state into a single state vector (h_t)
    - Removes output gate and returns state vector h_t at each time instant.
    - Implemented in ‘tf.keras.layers.GRU()’.

Kyunghyun Cho et al., “Learning Phrase Representations Using RNN Encoder-Decoder for Statistical Machine Translation,” Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (2014): 1724–1734.

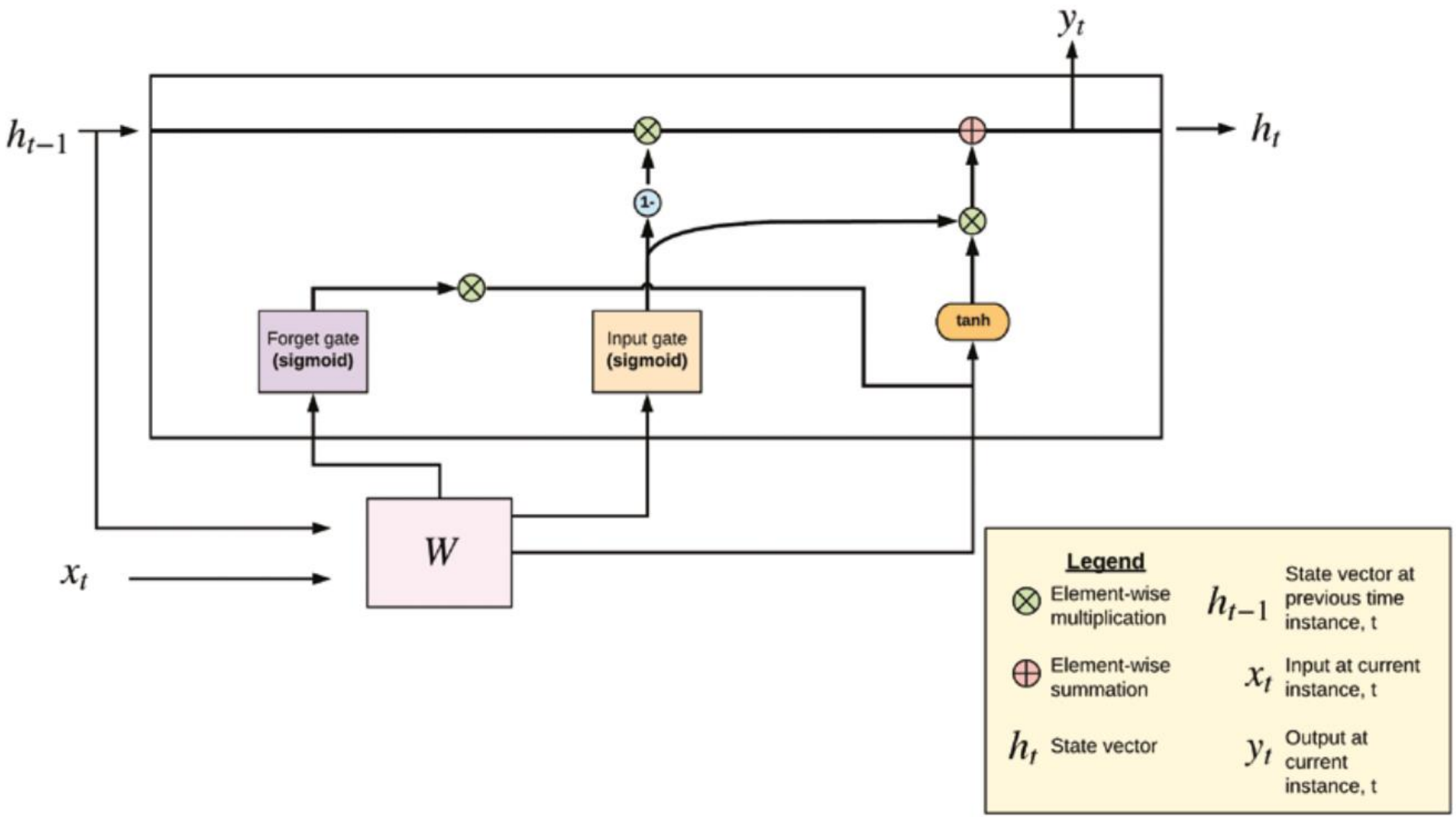
LSTM VS GRU





$1-Z_t$ (Z_t is the update gate output) represents the retention mechanism for the previous hidden state. Determines how much of the old information should be kept in the current hidden state.

Gated Recurrent Unit (GRU)



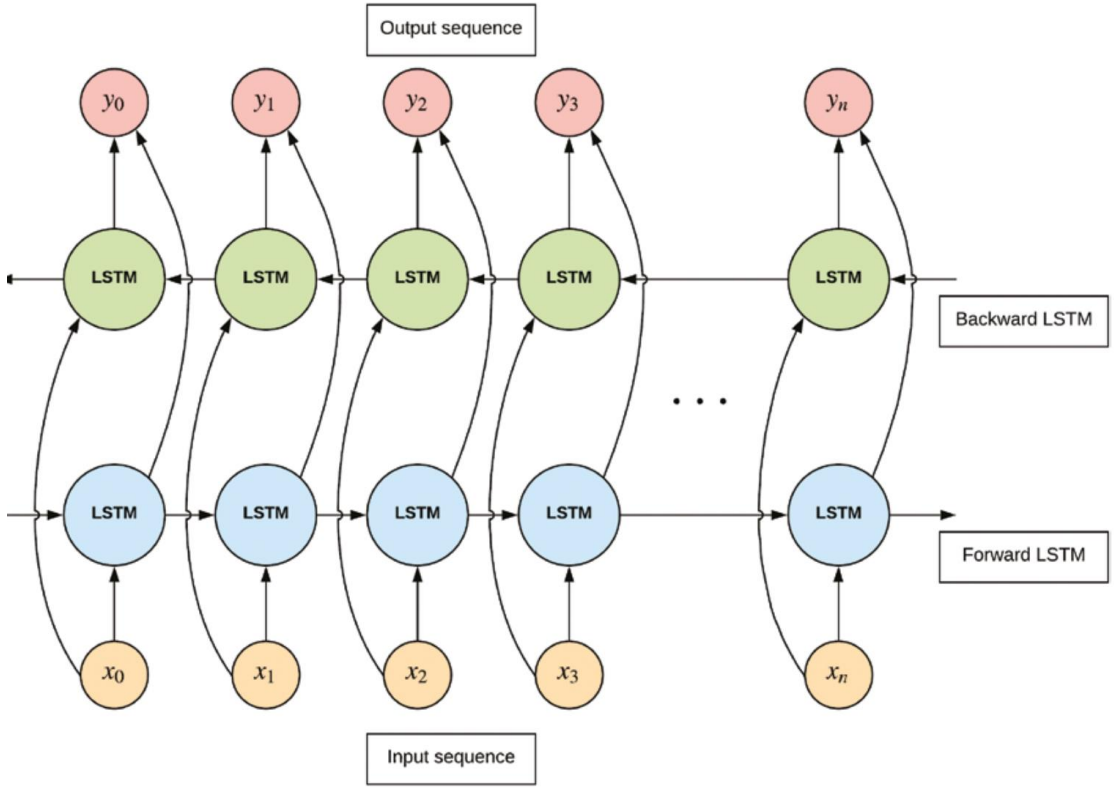
Building Machine Learning and Deep Learning Models on Google Cloud Platform By E. Bisong.

Bidirectional LSTM (BLSTM)

- A type of RNN that extends the standard unidirectional LSTM.
  - Improve sequence modeling performance by incorporating information from both temporal directions.
- BiLSTM: Combines the outputs of both networks
  - Uses one LSTM layer to process the sequence forward (past → future)
  - Uses another LSTM layer to process the sequence backward (future → past)
- BiLSTMs are widely used in:
  - Natural language processing
  - Speech processing
  - Sequence labeling
  - Time-series prediction
- BiLSTMs are particularly effective when:
  - The entire sequence is available at training and inference time
  - Context from both past and future improves prediction

Bi-LSTM

- **Forward LSTM** processes the input sequence in its natural temporal order:
  - The sequence is fed from the first-time step to the last (past → future).
  - Network learns long-term dependencies based on past information.
  - Each hidden state captures information from previous time steps.
- **Backward LSTM** processes the input sequence in reverse order:
  - The sequence is reversed and fed into the network (future → past).
  - Network learns dependencies based on future context.
  - Each hidden state captures information from subsequent time steps.



Img src: Building Machine Learning and Deep Learning Models on Google Cloud Platform By E. Bisong.

Comparison

	LSTM	GRU	Bi-LSTM
Architecture	Three gates: input, output, and forget	Two gates	Consist of two LSTM layers
Advantage	1. Capture long-term dependencies in sequential data 2. Handle complex sequences	1. Faster to train 2. Perform similarly to LSTMs, especially with less training data*	1. Offers comprehensive understanding 2. Better performance
Disadvantage	Complex structure	Cannot capture long-term dependencies	Computational complexity and training time

Summary

- Distributed Deep Learning: Parallelization strategies:
  - Data Parallelism – Replicate model; split data across workers; synchronize parameters.
  - Model Parallelism – Split model across workers; forward/backward propagation distributed.
  - Pipeline/Hybrid approaches

- Key challenges: Communication overhead + Synchronization + Scalability

- TensorFlow:
  - Open-source ML/DL framework
  - Computation expressed as dataflow graphs
    - Nodes → operations
    - Edges → tensors

Summary

- Recurrent Neural Networks (RNNs)
  - Designed for sequential and time-series data
  - Maintain hidden state (memory) across time
  - Trained using Backpropagation Through Time (BPTT)

- Key challenge: Vanishing & Exploding Gradients

- Advanced RNN Architectures
  - LSTM: Memory cell + input, forget, output gates
    - Captures long-term dependencies
  - GRU: Simplified LSTM (fewer gates)
    - Faster training, competitive performance
  - Bidirectional LSTM: Processes sequence:
    - Forward (past → future)
    - Backward (future → past)
    - Uses full context for prediction